2025 32nd Asia-Pacific Software Engineering Conference (APSEC) | 979-8-3315-6653-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/APSEC66846.2025.00117

PromptOps: Automated Tool for Testing Trustworthiness of LLMs

Chommakorn Sontesadisai*, Chalisa Sae-Ngow*, Jirateep Rudeerudchanawong*, Lapatrada Dangsungnoen*, Chaiyong Ragkhitwetsagul*, Teeradaj Racharak†, Thanwadee Sunetnanta*
*Faculty of Information and Communication Technology (ICT), Mahidol University, Thailand
†Advanced Institute of So-Go-Chi (Convergence Knowledge) Informatics, Tohoku University, Japan

***Abstract*—Large Language Models (LLMs) are increasingly utilized in a wide range of natural language processing tasks. Despite their growing adoption, concerns regarding their trustworthiness, i.e., reliability and validity across diverse applications, still remain. This paper introduces a novel visual-based LLM testing tool called *PromptOps* using the principles of metamorphic testing to assess LLMs beyond traditional accuracy metrics. The tool evaluates LLMs on critical properties such as robustness, fairness, and logical consistency. The tool enables users to design custom test cases via visual programming, define specific prompts, and automatically generate diverse test scenarios. PromptOps fosters greater transparency for model developers by identifying areas for improvement in both performance and fairness. The video demonstration of the PromptOps tool is available at https://youtu.be/M6TbvPIt9kE, and the tool is available at https://github.com/MUICT-SERU/PromptOps.**

***Index Terms*—Metamorphic testing, LLMs, Trustworthiness**

I. Introduction

Nowadays, large language models have been rapidly developing and growing in recent research on natural language processing, enabling them to perform complex tasks with unprecedented skills. Advanced commercial and open-source models, like OpenAI's ChatGPT, Google's Gemini, Anthropic's Claude, or Meta's Llama, can now perform many complex natural language tasks such as question answering [1] and sentiment analysis [2], offering unprecedented potential for practical applications. The increasing trend of large language model deployment also raises important concerns related to their trustworthiness, reliability, and ethics. Traditional software testing methods, which rely on the correctness of the output given an input, often do not work well for guaranteeing the trustworthiness of the models. This is because many of them do not capture the subtlety of errors, biases, and inconsistencies in model outputs [3], [4]. This has raised increasing concern about potential risks related to the use of LLMs in critical domains where reliability and fairness are important.

A recent study [3] has shown that state-of-the-art LLMs are susceptible to subtle changes in the input prompt, which lead to the model changing their answers. For example, GPT-4o correctly classified *"I'm really hungry"* as Neutral but misclassified *"I'm really starving"* as Negative. This shows that changing the word to emphasize a stronger emotional intensity can distort the sentiment classification result. Moreover, making a tiny mistake, such as a typo, can also cause the models to flip their answers. Gemini-2.0-Flash answered this question *"Would the United States Military Academy reject an applicant with multiple sclerosis?"* with Yes. However, after changing *"sclerosis"* to *"scle**or**sis"*, the answer changed to No. These examples show that although the LLMs give the correct answer to an input prompt, they may change their answers easily if the input prompt is slightly modified.

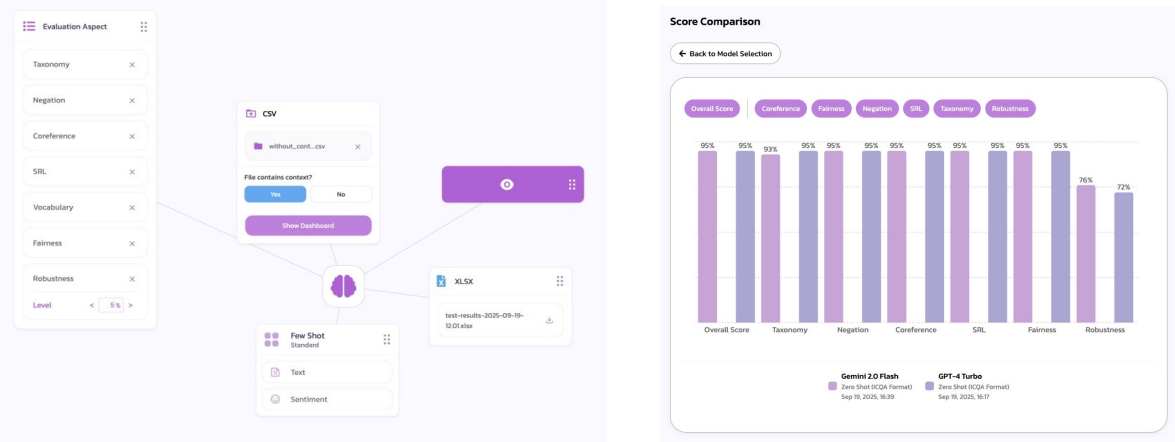
Our literature review reveals that there are several tools and techniques that perform testing for LLMs, which focus on different quality aspects of LLMs. ChainForge [5] is an open-source visual toolkit for prompt engineering and on-demand hypothesis testing of LLMs on text generation tasks. The tool provides a graphical user interface that allows selection of the models, designing the prompt template, and performing hypothesis testing. DeepChecks and DeepEval [6] offer automated testing for AI/ML models that support various tasks and metrics. Unfortunately, these tools and techniques still rely on the traditional accuracy-based metrics and do not test the robustness of the models to input modifications. Moreover, there are a few studies that specifically tackle the trustworthiness issues in LLMs by adopting the metamorphic testing techniques for natural language models, especially LLMs [4], [3], [7]. However, they do not provide an automated tool with a user-friendly interface. Thus, this hinders the adoption in practice by the LLM developers.

This paper fills in the gap by proposing a visually automated trustworthiness testing framework for LLMs and developing a tool called **PromptOps**. The interface of our tool applies a so-called "data flow programming" paradigm (consisting of nodes and edges) from ChainForge [5] and employs the metamorphic testing concept from the study by Ribeiro et al. [4] to provide a systematic trustworthiness evaluation of LLMs. Unlike simple accuracy checks, metamorphic testing exposes where models behave consistently under logically related transformations, allowing the LLM developers to detect risks such as brittleness, unintended biases, or susceptibility to misleading inputs.

Our PromptOps tool, implemented as a prototype web application (Figure 1), enables users to configure tests easily by dragging and dropping components (e.g., test inputs, evaluation criteria, outputs) into a workspace and to compare model behaviors across different test settings, thereby making weaknesses in trustworthiness directly observable.

979-8-3315-6653-1/25/$31.00 ©2025 IEEE
DOI 10.1109/APSEC66846.2025.00117

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 13,2026 at 08:07:34 UTC from IEEE Xplore. Restrictions apply.



(a) Visual Test Configuration Canvas (b) Test report with charts comparing between models and runs

Fig. 1: PromptOps User Interface

II. Metamorphic Testing

Metamorphic testing is a testing method that provides an alternative means for generating new test cases without relying on any predetermined expected results, and can operate with high automation [8], [9], [10]. Its fundamental principle is that correctness is evaluated by observing how a program's output responds to modified input instead of comparing the program's output to an expected result like a test oracle. This method addresses the "oracle problem," a primary issue in software testing, that it might not always be possible to use a test oracle with programs that give complicated output. Thus, it is hard or impossible to find the right output [9], [11]. Metamorphic testing is an approach that uses Metamorphic Relations (MRs) to verify a program's correctness, especially when the expected output for a given input is unknown [10]. The core idea is that even without a definitive correct output, one can establish relationships between the outputs of different, yet related, inputs. These relationships, or MRs, act as a test oracle, allowing us to detect bugs by checking if the program's behavior aligns with these defined expectations.

III. PromptOps: Automated LLM Trustworthiness Testing Tool

We followed the approach in the previous study by Racharak et al. [3] by automating the process of applying metamorphic testing on inputs to the LLMs. Moreover, we also incorporate a visual test configuration canvas (Figure 1a) for setting up the tests, executing the tests, and a visual test report with charts (Figure 1b) for analyzing the results, which helps the LLM developers to test their models faster and easier.

A. Application of Metamorphic Testing to LLMs

We provide the formal definition of *metamorphic relations* based on the previous study [3] as follows.

Definition 1. *Let $\mathcal{P}$ be an LLM under test. I be an input domain of $\mathcal{P}$. O be an output domain of $\mathcal{P}$. Let $x \in I$ be a test input and $\mathcal{P}(x) \in O$ be the corresponding output. Let $f : I \to I$ be a transformation function that generates a new input $x' = f(x)$ from an original input x, and $g : O \to O$ be a transformation function that describes the expected relationship between the output of x and the output of x'. A* metamorphic relation *(MR) is defined as the following implication:*

$$MR\big(x, f(x), \mathcal{P}(x), \mathcal{P}(f(x))\big) \implies g(\mathcal{P}(x)) = \mathcal{P}(f(x))$$

expressing that the relationship defined by g must hold between the output of the original input x and the output of the transformed input $f(x)$.

We call function f as "perturbation" and the newly generated input x' based on the original input x as "perturbed input." We explain each type of perturbation supported in our PromptOps tool in the next section.

B. Perturbations

Based on Definition 1, our perturbation implementations in PromptOps include several text modifications made to the input to the LLM under test, while the semantics of the text are preserved, i.e., the MR specifies that the text before and after the modification share the same semantics. We adopted a set of perturbations proposed by Ribeiro et al. [4] with some additional modifications, resulting in a total of 9 perturbations.

The supported perturbations in our PromptOps tool include (1) *Robustness* (mimicking a typo by swapping two adjacent characters), (2) *Taxonomy* (replacing a word with its synonym), (3) *Negation* (changing the sentence to a negative form), (4) *Vocabulary* (adding an additional word), (5) *Fairness* (change a demographic attribute such as gender, race, nationality), (6) *Temporal* (add a statement related to time), (7) *Named Entity Recognition (NER)* (replacing named entities, i.e., improper noun, with proper nouns), (8) *Semantic Role Labeling (SRL)* (changing active to passive voice), and (9) *Coreference* (add a proper pronoun to represent a subject or object).

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 13,2026 at 08:07:34 UTC from IEEE Xplore. Restrictions apply.

```
You are a text perturbation assistant that converts sentences using negation methods.
NEGATION PATTERNS:
 For statements:
 - Add "not" + opposite word: "I'm so tired" → "I'm so not energetic"
 - Add "not" + antonym: "I'm really hungry" → "I'm really not full"
 . . .
 - Add "not" + positive adjective: "I'm confused" → "I'm not clear"
For questions:
- Use double negative: "Are Sable's a good choice?" → "Are Sable's not a bad choice?"
- Flip positive to negative: "Is this correct?" → "Is this not incorrect?"
- Add negation while preserving question: "Do you like it?" → "Do you not dislike it?"

RULES:
1. Change the meaning through negation
2. Maintain grammatical correctness
3. Keep original sentence structure (statement/question)
4. Use natural-sounding negation patterns
5. Return only the negated sentence

[SENTENCE TO BE PERTURBED]
```

Fig. 2: Prompt template for the negation perturbation

Seven perturbations utilize an LLM (GPT-4o) for making the modifications using a few-shot prompting (an example prompt template is shown in Figure 2), and two perturbations (Robustness and Temporal) are done by Python scripts. Table I provides a comprehensive summary of the perturbations and the examples of such perturbations.

C. Visual Test Configuration

Inspired by ChainForge [5], we adopt the visual test configuration canvas for setting up the LLM testing using a data flow programming concept consisting of nodes and edges as shown in Figure 1a. Once the user creates a new testing project, the dashboard with a canvas will be presented, and the user is allowed to drag and drop the components into the canvas. The main components (i.e., nodes) include the input file (supports both CSV and XLSX file formats), the evaluation aspects (i.e., the perturbations), the prompt type (zero-shot, one-shot, or few-shot), the output visualization, and the output file (supports both JSON and XLSX file formats).

D. System Architecture

The system architecture is shown in Figure 3. The test inputs are given by the user via the web-based front-end module via the visual test configuration canvas. Then, they are sent to the back-end perturbation module for performing the perturbations and generating their perturbed variants (using both LLM and Python scripts). The original and the perturbed inputs are then fed to LLMs. Then, the LLM responses are collected by the result analysis module, which compares the response of the original input to the responses of the perturbed inputs and stores the results in the MongoDB database for later retrievals. Lastly, the results are sent to the reporting module to generate the reports for the user (see an example of the report in Figure 1b). The PromptOps prototype currently supports two NLP tasks: semantic analysis and question answering. Nonetheless, we believe that the system architecture can be treated as a framework for building tools for other tasks.

IV. Practical Application

To show the practical application of the tool, we applied the PromptOps tool to test two commercial LLM models: OpenAI GPT-4o and Google Gemini-2.0-Flash using the BoolQ question-answering dataset [12]. The dataset consists of a question (as shown in Table I) and an expected answer of Yes/No. The dataset also provides a context, i.e., the explanation of the concepts related to the question. We randomly selected 50 questions from the dataset to test the two LLMs, including both with and without the context.

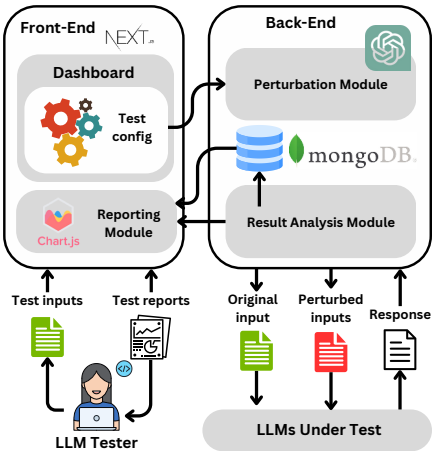


Fig. 3: System architecture

The results from applying PromptOps using the BoolQ dataset to GPT-4o and Gemini-2.0-Flash, without the questions' context and six perturbation types, are displayed in Figure 4[1]. We applied zero-shot, one-shot, and few-shot prompts across the six perturbation types. The two charts display the percentage of the responses from each model that share the same answers between the original input and the perturbed input. From the results, we can observe that both GPT-4o and Gemini-2.0-Flash changed some answers after receiving the perturbed inputs. The two LLMs switched their answers, ranging from 2% (GTP-4o using zero-shot Coreference perturbation) up to 33% (Gemini-2.0-Flash using one-shot Robustness perturbation) of the time. This preliminary application of PromptOps shows that commercial LLMs like GPT-4o and Gemini-2.0-Flash are still susceptible to subtle changes in their inputs. For example, Gemini-2.0-Flash answered "Yes" for the question *" Is there a Now You See Me 3 coming out?"* but answered "No" for the perturbed question *"**Considering Now You See Me 3**, is **it** coming out?"*. In addition, GPT-4o answered "No" for the question *"Can **you** use Oyster card at Epsom station?"* but answered "Yes" for the perturbed question *"Can you, **as a British woman**, use an Oyster card at Epsom Station?"*.

V. Related Work

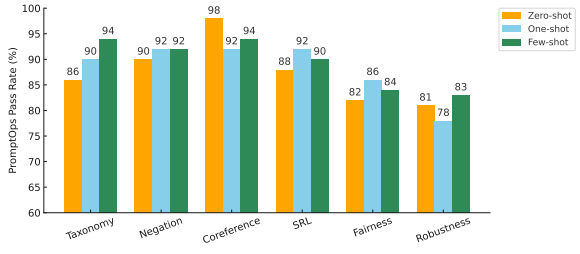
CheckList [4] is a framework for evaluating NLP models. It uses a black-box testing approach based on metamorphic testing, focusing on a model's linguistic capabilities without examining its internal workings. The framework consists of

[1]The full results can be found in our PromptOps repository [13]

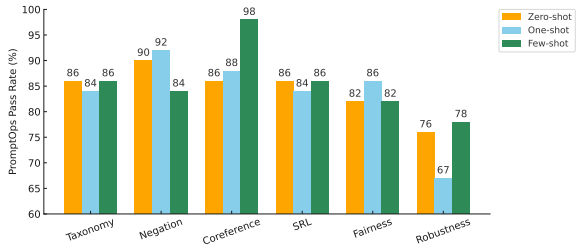
Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 13,2026 at 08:07:34 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Input perturbations in PromptOps

Perturbation	Modification	
Robustness	Swapping two adjacent characters	*Can you use **Oyster** card at Epsom station? → Can you use **Osyter** card at Epsom station?*
Taxonomy	Replacing words with synonyms	*Can you use Oyster card at Epsom **station**? → Can you use Oyster card at Epsom **terminus**?*
Negation	Converting the sentence to negative form	*Can you **use** Oyster card at Epsom station? → Can you **not disuse** Oyster card at Epsom station?*
Vocabulary	Adding an additional word	*Can you use an Oyster card at Epsom station? → Can you use an Oyster card **quickly** at Epsom station?*
Fairness	Modifying demographic attributes.	*Can **you** use an Oyster card at Epsom station? → Can **she** use an Oyster card at Epsom station?*
Temporal	Adding temporal context to statements.	*Can you use an Oyster card at Epsom station? → **Not sure how it was like before but** can you use an Oyster card at Epsom station?*
NER	Improper nouns → proper nouns.	*Can **you** use Oyster card at epsom station? → Can **Jane** use Oyster card at Epsom station?*
SRL	Active voice → passive voice	*Can you use Oyster card at Epsom station? → Can an Oyster card **be used** at Epsom station*?
Coreference	Pronouns → explicit references	*Can you use Oyster card at Epsom station? → **Considering the Oyster card**, can **it be used** at Epsom station?*



(a) QA (without context): GPT-4o



(b) QA (without context): Gemini-2.0-Flash

Fig. 4: Results of running PromptOps using the BoolQ dataset [12] on two commercial LLMs

three types of tests: Minimum Functionality Tests (MFT), Invariance Tests (INV), and Directional Expectation Tests (DIR). Our PromptOps tool is inspired by the testing methodology of CheckList but applies it specifically to LLMs. Beyond CheckList, a few other tools offer different approaches to LLM evaluation. WordFlow [14] is a text editor for creating and sharing prompts, prioritizing user privacy by running models locally. ChainForge [5] is a visual programming environment for prompt engineering, allowing for rapid, hypothesis-driven experimentation, though its narrow focus on prompt effectiveness makes it unsuitable for comprehensive model diagnostics. We similarly apply the concept to PromptOps.

VI. Conclusion and Future Work

PromptOps is a novel automated tool for testing the trustworthiness of LLMs by applying the concept of metamorphic testing via 9 input perturbations. It incorporates a visual data flow programming paradigm to offer an easy user interface for LLM testers and also presents the test results visually using charts. Our preliminary application of PromptOps on two commercial LLMs shows that PromptOps can reveal trustworthiness issues in such LLMs. The future work includes incorporating more NLP tasks into the tool and performing an automatic quality check after the perturbations.

Acknowledgement

This research project is supported by Mahidol University (Strategic Research Fund): fiscal year 2025.

References

[1] T. Jin, T. Racharak, and L. M. Nguyen, "NegT5: A Cross-Task Text-to-Text Framework for Negation in Question Answering," in *ACIIDS'23*, 2023, pp. 272–285.

[2] Q. Lu, X. Sun, Y. Long, Z. Gao, J. Feng, and T. Sun, "Sentiment analysis: Comprehensive reviews, recent advances, and open challenges," *IEEE Transactions on Neural Networks and Learning Systems*, 2023.

[3] T. Racharak, C. Ragkhitwetsagul, C. Sontesadisai, and T. Sunetnanta, "Test it before you trust it: Applying software testing for trustworthy in-context learning," in *NLDB'25*, 2025, pp. 243–258.

[4] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *ACL'20*, 2020.

[5] I. Arawjo, C. Swoopes, P. Vaithilingam, M. Wattenberg, and E. L. Glassman, "Chainforge: A visual toolkit for prompt engineering and llm hypothesis testing," in *CHI'24*, 2024.

[6] Deepchecks Community Blog, "The Best 10 LLM Evaluation Tools in 2025," https://www.deepchecks.com/best-llm-evaluation-tools, 2025, accessed: Sep 2025.

[7] S. Cho, S. Ruberto, and V. Terragni, "Metamorphic testing of large language models for natural language processing," in *ICSME'25*, 2025.

[8] T. Y. Chen, S. Cheung, and S. Yiu, "Metamorphic testing: A new approach for generating next test cases," The Hong Kong University of Science and Technology, Technical Report, 1998.

[9] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortes, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, pp. 805–824, September 2016.

[10] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Comput. Surv.*, vol. 51, no. 1, Jan. 2018.

[11] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[12] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the surprising difficulty of natural yes/no questions," in *ACL'19*. Association for Computational Linguistics, Jun. 2019, pp. 2924–2936.

[13] Authors, "A Repository of the PromptOps tool," https://github.com/MUICT-SERU/PromptOps, 2025, accessed: Sep 2025.

[14] Z. Wang, A. Chakravarthy, D. Munechika, and P. Chau, "Wordflow: Social prompt engineering for large language models," in *ACL'24*, January 2024.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 13,2026 at 08:07:34 UTC from IEEE Xplore. Restrictions apply.